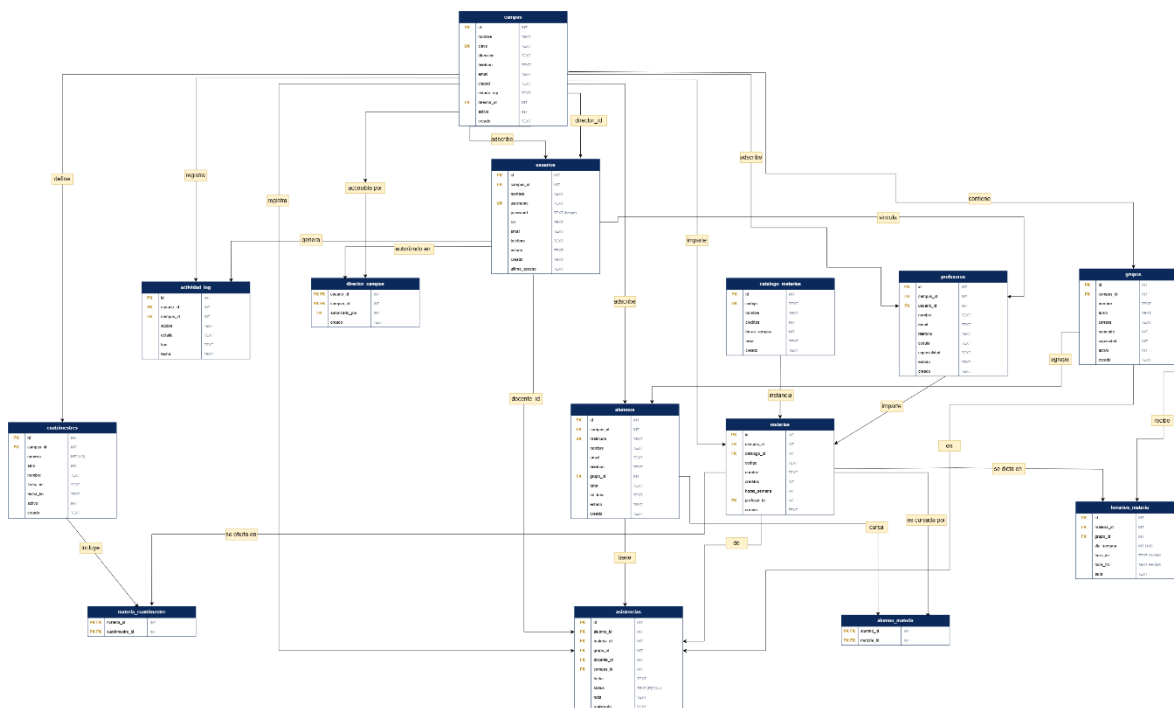


## Modelo de datos

Assisto usa 14 tablas relacionadas sobre SQLite. El esquema se crea automáticamente en el primer arranque desde server/db/database.js.

## Diagrama entidad-relación



## Decisiones de diseño

**Catálogo institucional separado de la instancia por campus.** catalogo\_materias guarda la definición única de cada asignatura (código, nombre, créditos, área). materias es la instancia concreta de esa asignatura en un campus, con su propio docente asignado. Así, SIS-301 existe una vez a nivel institucional pero puede impartirse en cuatro planteles con cuatro profesores distintos, sin duplicar la definición. La restricción UNIQUE(campus\_id, codigo) impide que un mismo código se repita dentro de un campus.

**Los horarios son una entidad, no columnas.** Una materia puede dictarse en varios bloques semanales. Modelarlo como columnas (dia\_1, hora\_1, dia\_2...) impondría un límite arbitrario y dejaría campos vacíos. horarios\_materia permite N bloques por materia y grupo, con UNIQUE(materia\_id, grupo\_id, dia\_semana, hora\_ini) para evitar duplicados exactos.

**Permisos de campus como tabla puente.** usuarios.campus\_id define la adscripción de un usuario, pero director\_campus permite que el superadministrador autorice a un director a ver planteles adicionales. Sin esa tabla, el alcance de un director estaría limitado a uno solo. La llave primaria compuesta (usuario\_id, campus\_id) impide autorizaciones duplicadas.

**Una asistencia por alumno, materia y fecha.** UNIQUE(alumno\_id, materia\_id, fecha) es la regla de negocio más importante del esquema: un alumno no puede tener dos registros contradictorios para la misma clase el mismo día. El backend aprovecha esto con INSERT OR REPLACE, de modo que corregir un pase de lista sobrescribe en lugar de duplicar.

**Desnormalización deliberada en asistencias.** campus\_id y grupo\_id son derivables desde alumno\_id, pero se almacenan igualmente. Es una desnormalización a propósito: los reportes y el dashboard filtran constantemente por campus y grupo, y tenerlos en la misma fila evita dos JOIN en cada consulta. Los índices idx\_asist\_campus e idx\_asist\_materia refuerzan ese patrón de acceso.

**Estados como CHECK, no como tablas de catálogo.** rol, estado, turno y status usan restricciones CHECK con valores literales en lugar de tablas relacionadas. Para conjuntos cerrados y pequeños que no cambian (cuatro estatus de asistencia, tres roles), una tabla de catálogo agregaría un JOIN sin aportar flexibilidad real.

**Borrado en cascada según dependencia.** ON DELETE CASCADE donde el registro hijo no tiene sentido sin el padre (asistencias de un alumno eliminado, materias de un campus eliminado). ON DELETE SET NULL donde el hijo sobrevive (una materia sin profesor asignado sigue existiendo; un alumno sin grupo también).

### Índices

|                    |                            |
|--------------------|----------------------------|
| idx_asist_fecha    | ON asistencias(fecha)      |
| idx_asist_materia  | ON asistencias(materia_id) |
| idx_asist_alumno   | ON asistencias(alumno_id)  |
| idx_asist_campus   | ON asistencias(campus_id)  |
| idx_alumnos_grupo  | ON alumnos(grupo_id)       |
| idx_alumnos_campus | ON alumnos(campus_id)      |

Todos apuntan a asistencias y alumnos, que son las tablas con más volumen y las que se consultan en cada carga del dashboard y cada reporte.

### Puntos pendientes del esquema

- **Referencia circular** entre campus.director\_id y usuarios.campus\_id: cada tabla apunta a la otra. SQLite lo tolera, pero obliga a insertar campus antes que usuarios y luego actualizar. Podría resolverse dejando la dirección del campus únicamente en director\_campus.
- **director\_campus está definida dos veces** en database.js: una en el esquema inicial y otra en el bloque de migraciones. La segunda es redundante.
- **alumno\_materia se crea también desde endpoints** en server/index.js, en lugar de vivir solo en el esquema.